Yuheng Li
Student ID: 23307130334

GitHub: CV_Midterm
ModelScope: cv-hw2-final-model

CV Assignment 2 Presentation

What This Presentation Explains

Classification

How an RGB pet image becomes a 37-class breed prediction.

Segmentation

How a pixel mask is produced and optimized with CE/Dice/Focal losses.

Detection + Tracking

How VisDrone boxes become YOLO predictions, object IDs, and line counts.

Thesis

Each extension is evaluated by asking: where is the bottleneck, what exactly was modified, and did the modification match the bottleneck?

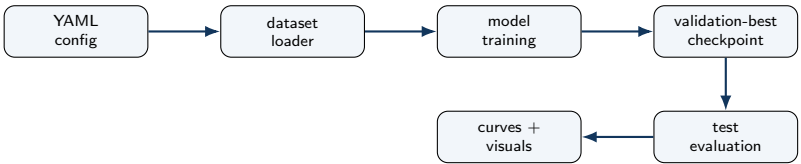
Final Results First

Task	Final model	Selection metric	Final result
Classification	ImageNet ResNet-34	validation accuracy	**90.60% test accuracy**
Segmentation	U-Net + Dice loss	validation mIoU	**77.90% test mIoU**
Detection	YOLO11m, img size 960	validation mAP50-95	**0.5356 mAP50 / 0.3310 mAP50-95**
Tracking	YOLO11m + ByteTrack	qualitative stability	line-crossing video analysis

Interpretation

The best gains came from pretrained representation, overlap-aware segmentation loss, and high-resolution dense detection.

Harness: Experimental Control, Not Decoration



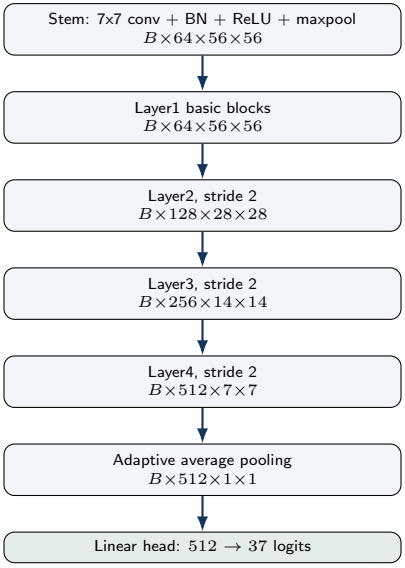
- Classification selects by validation accuracy; segmentation by validation mIoU; detection by validation mAP50-95.
- The test result is computed from the selected checkpoint, not from the last epoch.

Classification Data Flow

Training signal

Cross entropy compares the 37 logits with the category label. Accuracy is the fraction of images whose top-1 logit matches the ground-truth class.

ResNet Baseline Architecture From Code



Baseline modification

Start from torchvision ResNet-18. Replace only the original ImageNet FC layer with a new 37-class FC layer.

ResNet-18 vs 34

Spatial dimensions stay the same.
ResNet-34 only uses more basic blocks: $[3, 4, 6, 3]$ instead of $[2, 2, 2, 2]$.

Optimization

Backbone LR $= 10^{-4}$; head LR $= 10^{-3}$.
The new head is allowed to adapt faster.

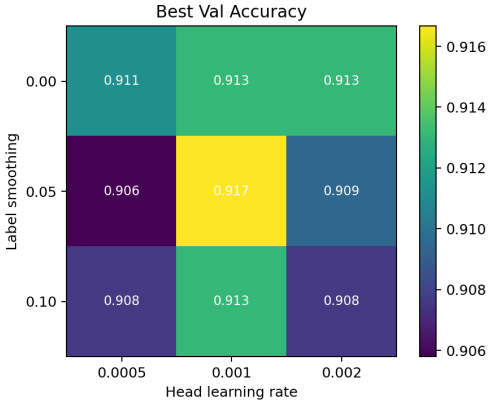
Classification Extensions: Where They Enter

Extension	Exact change	Difference from baseline
Random init	Use ResNet-18 with `weights=None`.	Same architecture, no ImageNet features.
ResNet-34	Replace ResNet-18 backbone with pretrained ResNet-34.	Deeper residual backbone, same 37-class head.
Label smoothing	Set `label_smoothing=0.1` in CrossEntropyLoss.	Target is softened instead of one-hot.
Freeze-unfreeze	Freeze backbone for first 3 epochs, then unfreeze all layers.	Early training updates only head parameters.

Common rule

Dataset, transforms, optimizer family, and validation-best selection stay fixed.

Classification Hyperparameter Grid Search



Controlled grid

Fixed pretrained ResNet-18, batch 32, image size 224, backbone LR 10^{-4}, weight decay 10^{-4}.

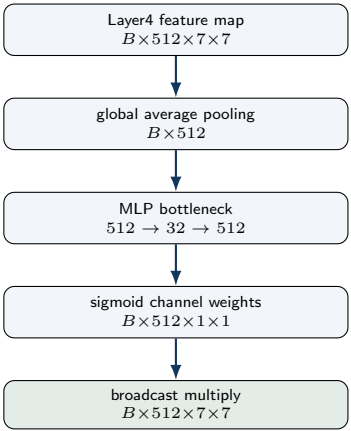
Swept hyperparameters

`label_smoothing`: 0, 0.05, 0.10.
`head_lr`: 5×10^{-4}, 10^{-3}, 2×10^{-3}.

Reading

Best validation setting is label smoothing 0.05 and head LR 10^{-3}, but gains are small.

SE Block: The Attention Extension



Insertion point

In code, SE is inserted after ResNet layer4 and before average pooling.

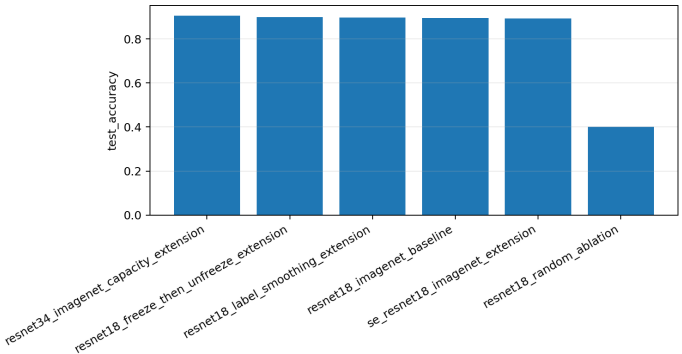
Purpose

It learns which channels should be emphasized for fine-grained breed cues.

Result

It did not beat the pretrained ResNet-18 baseline, so the added module was not automatically useful.

Classification Results



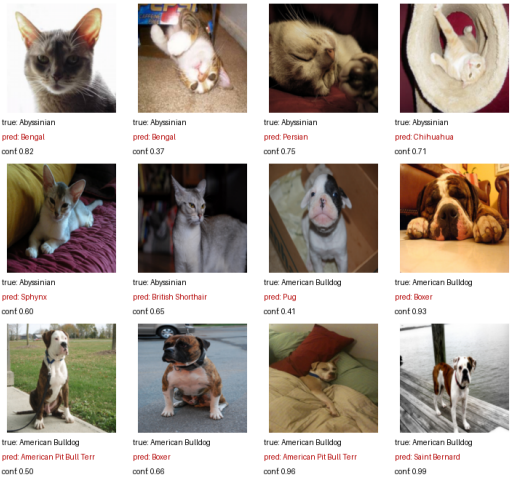
Key evidence

- Random ResNet-18: **40.15%**.
- Pretrained ResNet-18: **89.40%**.
- Pretrained ResNet-34: **90.60%**.

Reading

Transfer learning is the largest factor; extra capacity gives the best extension.

Classification Failure Cases

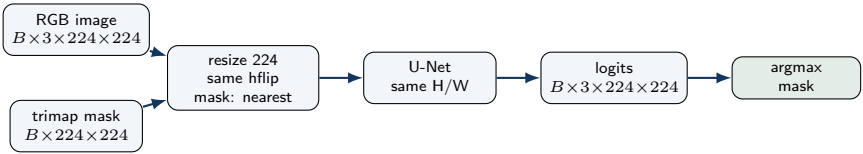


What the mistakes mean

The model usually fails between visually similar breeds, not between unrelated categories.

- Terrier/bulldog-like breeds share short fur and head structure.
- Ragdoll/Birman-like cats share color masks and coat patterns.
- This explains why representation capacity helps but does not remove all errors.

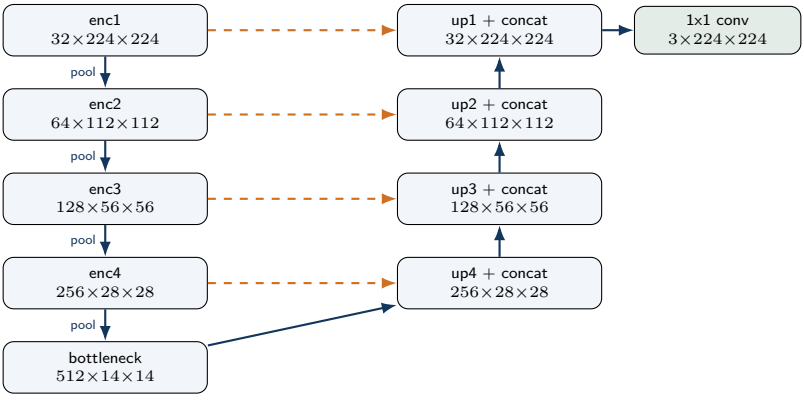
Segmentation Data Flow



Code-level detail

The original trimap values are clipped to 1–3 and shifted to 0–2. Thus the model predicts three classes: foreground, boundary, and background.

U-Net Architecture From Code



Each DoubleConv is Conv-BN-ReLU-Conv-BN-ReLU. Dashed arrows are skip concatenations.

How the Segmentation Losses Are Applied

Cross-Entropy

Uses raw 3-channel logits and integer mask labels. Each pixel is treated as a 3-way classification problem.

Dice

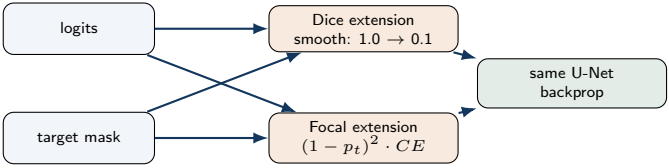
Applies softmax to logits, converts target to a one-hot mask, then computes overlap:

$$1 - \frac{2|P \cap Y| + s}{|P| + |Y| + s}$$

CE + Dice

Adds the two losses. CE stabilizes pixel classification; Dice aligns with region overlap and mIoU.

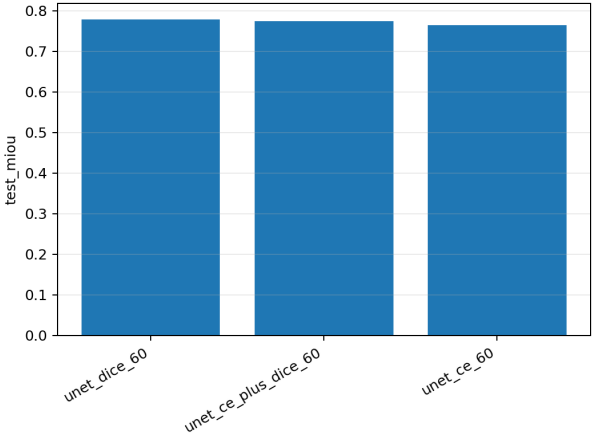
Segmentation Extensions: Loss-Level Changes



Important control

The architecture, image size, optimizer, and 60-epoch budget are unchanged; only the criterion changes.

Segmentation Results



Required comparison

CE: 76.40%; Dice: **77.90%**; CE+Dice: 77.41%.

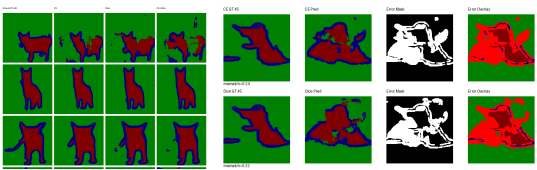
Extensions

Dice smooth 0.1: 77.85%; Focal: 76.45%.

Reading

Dice is best because the evaluation metric is overlap-based.

Segmentation Qualitative Explanation



What remains hard

The coarse pet region is usually correct, but boundary pixels remain unstable.

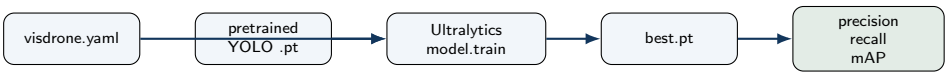
- Thin legs, ears, and fur edges are hard.
- Low-contrast foreground/background regions are ambiguous.
- This explains why Focal Loss does not automatically improve mIoU.

VisDrone Label Conversion

Dataset output

The converter creates symlinked image folders, YOLO-format label files, and a `visdrone.yaml` file with 10 classes.

YOLO Detection Training Pipeline



- Baseline: YOLOv8n, image size 640.
- Capacity extension: YOLOv8s, same data and same image size.
- Strong extension: YOLO11m with image size 960.

YOLOv8n vs YOLOv8s vs YOLO11m

Model	Main blocks	Scale / size	Parameters
YOLOv8n	C2f + SPPF + Detect	width 0.25	3.16M
YOLOv8s	C2f + SPPF + Detect	width 0.50	11.17M
YOLO11m	C3k2 + SPPF + C2PSA + Detect	scale m	20.11M

Model	imgsz	mAP50	mAP50-95
YOLOv8n	640	0.2985	0.1665
YOLOv8s	640	0.3819	0.2211
YOLO11m	960	**0.5356**	**0.3310**

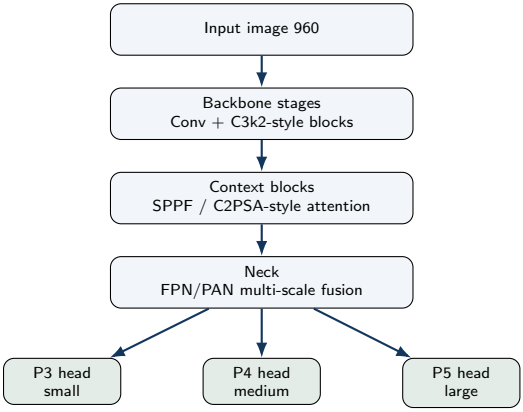
Controlled read

v8n → v8s keeps 640 input, so the gain mainly reflects detector capacity.

Practical read

YOLO11m@960 changes both model family and resolution, so it is a stronger practical detector rather than a single-factor ablation.

YOLO11m Architecture: More Detailed View



What I changed

I did not edit YOLO11m internals. I selected YOLO11m and increased input size to 960.

Why for VisDrone

Small objects occupy more pixels at 960 input, and multi-scale heads help dense objects at different sizes.

Source: Ultralytics YOLO11 docs,

`https://docs.ultralytics.com/models/yolo11/`

Detection Extensions: What Changed

Experiment	Exact modification	What it tests
YOLOv8n baseline	`model=yolov8n.pt`, `imgsz=640`.	Lightweight single-stage baseline.
YOLOv8s	Change only model size to `yolov8s.pt`; keep `imgsz=640`.	Whether more capacity helps.
YOLO11m@960	Change model to `yolo11m.pt`; increase `imgsz=960`.	Practical combination of newer capacity and higher small-object resolution.

Limitation

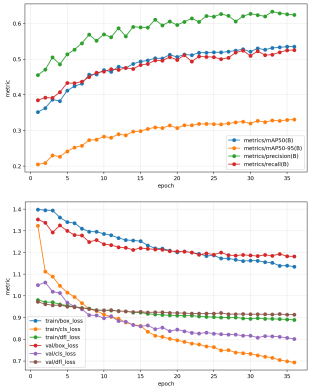
YOLO11m and 960 resolution are not fully disentangled, so the conclusion is about this practical combined extension.

Detection Results

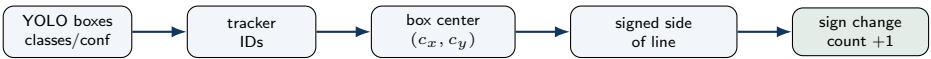
Model	Size	mAP50	mAP50-95
YOLOv8n	640	0.2985	0.1665
YOLOv8s	640	0.3819	0.2211
YOLO11m	960	**0.5356**	**0.3310**

Reading

VisDrone benefits strongly from larger capacity and higher resolution.



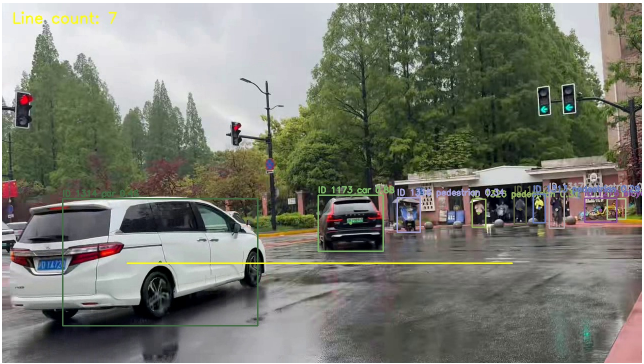
Tracking and Line-Crossing Algorithm



Counting rule

For each track ID, store which side of the line its center lies on. If the signed side changes sign and the ID has not been counted before, it is counted as one crossing.

Tracking Demo



Demo file

`presentation/videos/`

`yolo11m_bytetrack_c025.mp4`

- YOLO11m detects boxes.
- ByteTrack maintains object IDs.
- Confidence threshold changes which objects survive into tracking.

Fallback

The static frame shows IDs, line, and count if the video player fails.

Overall Discussion

Task	Bottleneck	What worked
Classification	fine-grained representation	ImageNet pretraining + ResNet-34
Segmentation	region overlap + boundary quality	Dice loss, but not Focal Loss
Detection	tiny dense objects	YOLO11m + 960 input
Tracking	ID stability under occlusion	lower-conf ByteTrack was more complete

Negative results

Random initialization, SE attention, Focal Loss, and high tracking confidence are useful because they show what did not match the main bottleneck.

Conclusion

- The project completed all three required tasks plus controlled extension studies.
- The best models were selected by validation metrics and tested from best checkpoints.
- Final results: 90.60% classification accuracy, 77.90% segmentation mIoU, and 0.3310 detection mAP50-95.
- Code, configs, report, and weights are released through GitHub and ModelScope.

Final message

A modification is useful only when it matches the real bottleneck of the task.

Appendix: Links and Backup

- GitHub: `https://github.com/yuheng-li-ai/CV_Midterm`
- ModelScope weights: `https://www.modelscope.cn/models/yuhengli/cv-hw2-final-model`
- YOLO11 documentation: `https://docs.ultralytics.com/models/yolo11/`
- Tracking video: `presentation/videos/yolo11m_bytetrack_c025.mp4`

Backup answer

YOLO11m@960 was stopped at epoch 36 due to deadline constraints, but it already had the strongest validation score among all detection runs.